uc3m | Universidad Carlos III de Madrid

Master's Degree in Applied and Computational Mathematics

2024-2025

Master's Thesis

"Conformable Fractional Physics-Informed Neural Networks"

David Specogna Acosta

Victor Bayona Revilla

Pedro Gonzalez Rodriguez

Madrid, September 2025

AVOID PLAGIARISM

The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.

This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

Physics-Informed Neural Networks (PINNs) have emerged as a powerful framework for solving integer-order differential equations, but their application to Fractional Differential Equations (FDEs) is challenged by the non-local, integral-based nature of classical fractional operators (e.g., Caputo, Riemann-Liouville), which is incompatible with Automatic Differentiation (AD). The state-of-the-art solution, the Fractional PINN (fPINN), circumvents this issue by using numerical discretization for fractional terms, but this approach introduces discretization error into the loss function and faces scalability limitations in high dimensions.

This thesis proposes a novel, fully differentiable framework: the Conformable Fractional PINN (cfPINN). We leverage the recent improved conformable derivative definition, which is local and AD-compatible, to construct a surrogate FDE model that is solved entirely within the PINN architecture. This approach presents a fundamental trade-off: it eliminates the discretization error of fPINNs in exchange for an inherent surrogate modeling error.

We implement the cfPINN framework in JAX and validate it on benchmark FDEs, including 1D initial value problems, 1D fractional Poisson problems, and a 2D time-dependent advection-diffusion equation. We demonstrate that for forward problems where sparse data from the true non-local solution is available, the cfPINN can learn an effective fractional order $\hat{\alpha}$ that allows the local model to accurately approximate the non-local dynamics. We conclude that the cfPINN is a viable alternative to fPINNs, offering a different set of trade-offs that may be advantageous, particularly for problems where high-dimensional scalability is critical.

Keywords: Physics-Informed Neural Networks, Fractional Differential Equations, Conformable Fractional Derivative, Automatic Differentiation, Scientific Machine Learning, Fractional Physics Informed Neural Networks, Deep Learning.

CONTENTS

1. INTRODUCTION . . . 1
1.1. Physics-Informed Neural Networks (PINNs) . . . 1
1.1.1. The Neural Network as a Surrogate Model . . . 2
1.1.2. The Physics-Informed Loss Function . . . 3
1.1.3. Training via Loss Minimization . . . 4
1.1.4. Advantages of PINNs . . . 5
1.2. Automatic Differentiation . . . 5
1.2.1. Forward Mode AD . . . 6
1.2.2. Backward Mode AD . . . 7
2. FRACTIONAL DERIVATIVES AND PHYSICS-INFORMED METHODS . . . 10
2.1. The Challenge of Non-Locality in Deep Learning . . . 10
2.2. fPINNs: The State-of-the-Art Methodology . . . 11
2.2.1. The Hybrid Loss Function . . . 11
2.2.2. Advantages and Limitations of the fPINN Approach . . . 12
2.3. Conformable Derivatives: An AD-Native Alternative . . . 12
2.3.1. The Conformable Derivative . . . 12
2.3.2. The Improved Conformable Derivative . . . 13
3. METHODOLOGY . . . 15
3.1. Surrogate Model Architecture and Modeling Error . . . 15
3.2. Forward problems . . . 16
3.3. Implementation in JAX . . . 17
3.3.1. One-Dimensional Operators . . . 17
3.3.2. Multidimensional Problems . . . 19
4. NUMERICAL SIMULATIONS . . . 21
4.1. One-Dimensional Fractional Initial Value Problem . . . 21
4.1.1. Problem Statement . . . 21
4.1.2. Methodology and Hyperparameters . . . 22
4.1.3. Results and Discussion . . . 22

4.2. One-Dimensional Fractional Poisson Problem 25
4.2.1. Problem Statement 26
4.2.2. Methodology and Hyperparameters 26
4.2.3. Results and Discussion 27
4.3. Two-Dimensional Time-Dependent Fractional Advection-Diffusion 28
4.3.1. Problem Statement 28
4.3.2. Methodology and Hyperparameters 29
4.3.3. Results and Discussion 29
5. SUMMARY AND CONCLUSIONS 32
BIBLIOGRAPHY. 33

LIST OF FIGURES

1.1 PINN Architecture Components 2

1.2 The forward mode evaluation trace for $f(x) = 3\sin(x^2)$ at $x = 2$. The top row shows the dependency of intermediate variables. The Primal trace evaluates the function value, while the Tangent trace concurrently evaluates the derivative. 7

1.3 Computational trace for $f(x, y) = \sin(x)\cos(y)$ at $(x, y) = (\pi/2, \pi/3)$. . . 8

4.1 Forward Problem (Fixed α): Comparison of the cfPINN solution to the exact analytical solutions. 23

4.2 Forward Problem (Fixed α): Training loss over iterations. 23

4.3 Forward Problem (Fixed α): L2 error relative to the improved conformable solution over training iterations. 24

4.4 Forward Problem (Learnable $\hat{\alpha}$): Convergence of the learned effective fractional order. 24

4.5 Forward Problem (Learnable $\hat{\alpha}$): The L2 relative error of the cfPINN solution versus the learned effective order during the training trajectory. . 25

4.6 Forward Problem (Learnable $\hat{\alpha}$): Comparison of the final cfPINN solution against the analytical solutions. The model accurately matches the true non-local Caputo solution. 25

4.7 Training dynamics for the Poisson problem: Convergence of the learned approximate order $\hat{\alpha}$. 27

4.8 Training dynamics for the Poisson problem: L2 error vs. the learned approximate order $\hat{\alpha}$. 27

4.9 Comparison of the final cfPINN solution with the exact analytical solution for the Poisson problem. 28

4.10 Convergence of the learned fractional order $\hat{\alpha}$ for the 2D problem. 30

4.11 The L2 relative error of the PINN solution plotted against the learned spatial order $\hat{\alpha}$. 30

4.12 Visual comparison of the final PINN solution against the analytical solution for the 2D problem at a time slice of $t = 0.5$. 31

LIST OF TABLES

4.1 Hyperparameters for the 1D Fractional IVP experiments. 22

4.2 Hyperparameters for the 1D Fractional Poisson problem with a learnable order. 26

4.3 Hyperparameters for the 2D time-dependent problem with a learnable spatial order. 29

5.1 A comparison of advantages (+) and disadvantages (-) of the proposed cfPINN method against the fPINN approach. 32

1. INTRODUCTION

This thesis begins by introducing the core concepts of Physics-Informed Neural Networks (PINNs) and Automatic Differentiation (AD). These topics form the foundation for the methods explored, as AD is particularly important in that it poses the main challenge in extending the approach to Fractional Differential Equations (FDEs).

The motivation for developing robust solvers for FDEs is compelling, as fractional calculus has become an indispensable tool for modeling complex systems with memory or non-local effects. Phenomena such as anomalous diffusion in disordered media, for instance, are ubiquitous in nature but cannot be adequately described by classical integer-order models; FDEs provide a robust mathematical framework for these processes (Metzler & Klafter, 2000). This descriptive power extends far beyond physics, with critical applications in diverse fields. In bioengineering, FDEs are used to model the viscoelastic properties of biological tissues, capturing the complex, history-dependent response of soft tissues to mechanical stress (Magin, 2010). In finance, fractional models are employed to describe long-range memory effects in financial market volatility and to develop more accurate option pricing models that account for the non-local dependencies observed in asset prices (Scalas, 2006). The growing importance of such models across science and engineering underscores the urgent need for efficient and accurate numerical solvers.

To address this need, Chapter 2 gives a brief overview of fractional calculus and reviews the state of the art in solving FDEs with PINNs, which is the fPINN method. Furthermore, it introduces the conformable fractional derivative, which is the basis of our proposed method (cfPINNs). This methodology is outlined in Chapter 3 where we also discuss its implementation in JAX and the trade-offs involved. Finally, we test this methodology in Chapter 4 by numerically solving several benchmark FDEs.

1.1. Physics-Informed Neural Networks (PINNs)

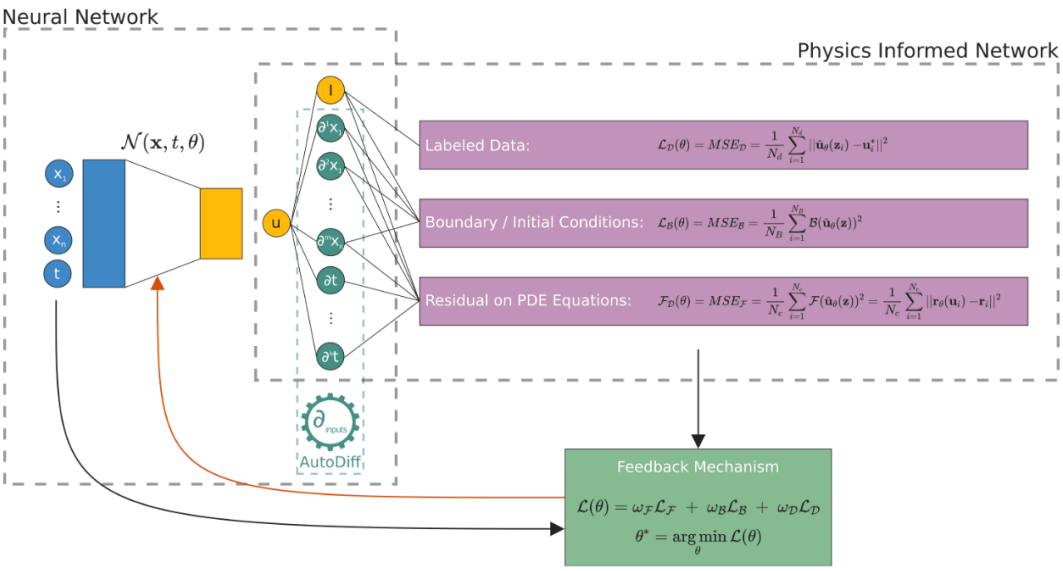
Physics-Informed Neural Networks (PINNs) represent a modern deep learning framework designed to solve differential equations by embedding physical laws directly into the training process of a neural network (Cuomo et al., 2022). This paradigm uses the expressive power of neural networks as universal function approximators, while enforcing the governing principles of a physical system via the loss function. The result is a mesh-free and versatile approach for both forward and inverse problems in differential equations.

The conceptual architecture of a PINN, illustrated in Figure 1.1, will guide our subsequent discussion of its constituent parts: (1) a neural network that serves as a surrogate for the unknown solution, (2) a physics-informed loss function that measures the network's compliance with the governing physical laws and data, (3) an optimization algorithm that trains the network by minimizing this loss and (4) the use of *Automatic Differentiation*

(AD). AD will be discussed in further detail in Chapter 1.2. The following sections will provide a mathematical description of each of these components, following the structure laid out in the diagram. Finally, we give a brief summary of the advantages of using PINNs over traditional numerical methods.

Figure 1.1

The core architecture of a Physics-Informed Neural Network. The framework is built upon a neural network surrogate, a composite loss function enforcing physical laws, and a feedback mechanism for optimization. Adapted from (Cuomo et al., 2022).



1.1.1. The Neural Network as a Surrogate Model

The first component of the PINN framework is a neural network that approximates the solution of the differential equation. This corresponds to the *Neural Network* block in Figure 1.1. We consider a general partial differential equation (PDE) of the form:

$$\mathcal{F}(\boldsymbol{u}(z); \boldsymbol{\gamma}) = \boldsymbol{f}(z), \quad z \in \Omega \subset \mathbb{R}^d \tag{1.1}$$

$$\mathcal{B}(\boldsymbol{u}(z)) = \boldsymbol{g}(z), \quad z \in \partial\Omega \tag{1.2}$$

where $z := [x_1, \ldots, x_{d-1}; t]$ are the spatio-temporal input coordinates, $\boldsymbol{u}$ is the unknown solution, $\boldsymbol{\gamma}$ represents physical parameters, $\mathcal{F}$ is a nonlinear differential operator, and $\mathcal{B}$ defines the boundary conditions.

The PINN methodology proposes a continuous and differentiable surrogate model $\hat{\boldsymbol{u}}_\theta(z)$ to approximate $\boldsymbol{u}(z)$, where θ represents the trainable parameters of the network. This surrogate is typically a Multilayer Perceptron (MLP), whose suitability is supported by the Universal Approximation Theorem. This theorem states that multilayer feed-forward networks can approximate any continuous function to an arbitrary degree of

accuracy[1](Hornik et al., 1989).

Definition 1.1.1 (Multilayer Perceptron (MLP))**.** *An MLP with an input layer, $K-1$ hidden layers, and an output layer is a function $\hat{\boldsymbol{u}}_\theta : \mathbb{R}^{d_{in}} \to \mathbb{R}^{d_{out}}$ defined by the recursive composition of functions:*

$$\hat{\boldsymbol{u}}_\theta(\boldsymbol{z}) = C_K \circ \sigma \circ C_{K-1} \circ \cdots \circ \sigma \circ C_1(\boldsymbol{z}) \tag{1.3}$$

where σ is a non-linear activation function (e.g., tanh*, ReLU, or sigmoid) applied element-wise, and C_k are affine transformations for each layer $k = 1, \ldots, K$:*

$$C_k(\boldsymbol{a}) = \boldsymbol{W}_k \boldsymbol{a} + \boldsymbol{b}_k \tag{1.4}$$

The set of all trainable parameters is $\theta = \{\boldsymbol{W}_k, \boldsymbol{b}_k\}_{k=1}^K$, where $\boldsymbol{W}_k$ are weight matrices and $\boldsymbol{b}_k$ are bias vectors. The network's architecture (number of layers and neurons per layer) is a critical hyperparameter.

This MLP provides a continuous representation of the solution over the entire domain Ω, which can be evaluated at any arbitrary point $\boldsymbol{z}$ without the need for a mesh.

1.1.2. The Physics-Informed Loss Function

The second component is the loss function, which guides the training process. As shown in the *Physics Informed Network* block of Figure 1.1, the total loss is a weighted sum of several distinct terms, each enforcing a different constraint on the solution.

Definition 1.1.2 (Composite Loss Function)**.** *The composite loss function[2] $\mathcal{L}(\theta)$ is defined as the weighted sum of the PDE residual, boundary condition, and data fidelity losses:*

$$\mathcal{L}(\theta) = \omega_{\mathcal{F}} \mathcal{L}_{\mathcal{F}}(\theta) + \omega_{\mathcal{B}} \mathcal{L}_{\mathcal{B}}(\theta) + \omega_d \mathcal{L}_{data}(\theta) \tag{1.5}$$

where $\omega_{\mathcal{F}}$, $\omega_{\mathcal{B}}$, and ω_d are user-defined weights. The constituent terms are defined as follows:

1. ***The PDE Residual Loss,*** *$\mathcal{L}_{\mathcal{F}}$, is the mean squared error of the PDE residual over a set of N_f collocation points $\{\boldsymbol{z}_f^i\}_{i=1}^{N_f}$ from the domain Ω:*

$$\mathcal{L}_{\mathcal{F}}(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} \|\mathcal{F}(\hat{\boldsymbol{u}}_\theta(\boldsymbol{z}_f^i)) - \boldsymbol{f}(\boldsymbol{z}_f^i)\|^2 \tag{1.6}$$

[1]The formal statement of the theorem is more general. As shown in (Hornik et al., 1989), it applies not only to continuous functions but also to the broader class of Borel measurable functions. Furthermore, while the theorem's power lies in guaranteeing universality with just a single hidden layer (given sufficient width), this approximation property naturally extends to the deeper multilayer architectures used in practice.

[2]From the perspective of the calculus of variations, this can be viewed as a loss functional, since it maps the entire candidate solution function $\hat{\boldsymbol{u}}_\theta$ to a scalar value. However, in alignment with standard deep learning terminology, we will refer to it as a loss function of the trainable parameters θ.

2. ***The Boundary Condition Loss,*** *$\mathcal{L}_{\mathcal{B}}$, is the mean squared error over N_b points $\{z_b^j\}_{j=1}^{N_b}$ on the boundary $\partial\Omega$:*

$$\mathcal{L}_{\mathcal{B}}(\theta) = \frac{1}{N_b} \sum_{j=1}^{N_b} \|\mathcal{B}(\hat{\boldsymbol{u}}_\theta(z_b^j)) - \boldsymbol{g}(z_b^j)\|^2 \tag{1.7}$$

3. ***The Data Fidelity Loss,*** *$\mathcal{L}_{data}$, is the mean squared error over N_d labeled data points $\{z_d^k, \boldsymbol{u}_k\}_{k=1}^{N_d}$:*

$$\mathcal{L}_{data}(\theta) = \frac{1}{N_d} \sum_{k=1}^{N_d} \|\hat{\boldsymbol{u}}_\theta(z_d^k) - \boldsymbol{u}_k\|^2 \tag{1.8}$$

1.1.3. Training via Loss Minimization

The final component of the framework is the training process, represented by the *Feedback Mechanism* in Figure 1.1. The goal is to find the optimal set of network parameters θ^* that minimizes the composite loss function:

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta) \tag{1.9}$$

This optimization problem is typically high-dimensional and non-convex. The solution process relies on gradient-based optimization, which is made possible by automatic differentiation (see Chapter 1.2). AD is the computational engine of the framework, serving two distinct roles. First, it is used to exactly compute the derivatives of the network's output $\hat{\boldsymbol{u}}_\theta(z)$ with respect to its inputs z, which is necessary to evaluate the differential operator $\mathcal{F}$ in the physics loss term. Second, it is used to compute the gradients of the total loss function $\mathcal{L}(\theta)$ with respect to all network parameters θ. This latter process is famously known as *backpropagation* (Rumelhart et al., 1986).

The gradients computed via backpropagation are then fed into sophisticated optimization algorithms to update the network parameters. Due to the challenging nature of the PINN loss landscape, a common and effective training strategy involves a two-phase approach (Cuomo et al., 2022):

- **Phase 1: Adam Optimizer.** Training is typically initiated with the **Adam** (Adaptive Moment Estimation) optimizer (Kingma & Ba, 2014). Adam is a stochastic gradient-based method that is robust and efficient for navigating complex loss landscapes in the early stages of training.

- **Phase 2: L-BFGS Optimizer.** After a set number of iterations, practitioners often switch to a quasi-Newton method, most commonly **L-BFGS** (Limited-memory Broyden–Fletcher–Goldfarb–Shanno) (Nocedal, 1980). L-BFGS is effective for fine-tuning the solution and achieving rapid convergence once the optimizer is near a local minimum.

This hybrid optimization strategy leverages the broad exploratory power of Adam and the precise convergence of L-BFGS to effectively train the network.

1.1.4. Advantages of PINNs

The PINN framework offers several distinct advantages over traditional numerical solvers:

- **Mesh-free formulation**: Collocation points can be sampled arbitrarily within the domain, making it easy to handle complex geometries and high-dimensional problems.
- **Unified framework**: The same architecture can solve both forward problems (finding the solution $\boldsymbol{u}$) and inverse problems (inferring parameters $\boldsymbol{\gamma}$ from data) by simply including the unknown parameters in the trainable set θ.
- **Continuous representations**: The neural network surrogate provides a solution that is differentiable and can be evaluated at any point in the domain, not just at discrete grid points.
- **Leverages modern hardware**: The methodology is built on deep learning libraries (like PyTorch and JAX) that are highly optimized for parallel computation on GPUs and TPUs.

1.2. Automatic Differentiation

Automatic differentiation (AD) is at the core of the PINN architecture. It is the method by which the information of the differential equation is translated into a training loss. Furthermore, it has wide uses in machine learning, and implementations of it are present in many popular libraries such as PyTorch (Paszke et al., 2019) and Jax (Bradbury et al., 2018). In this section we give a brief description of how AD functions and its relevance in machine learning and PINNs.

To understand the advantages of AD it is best to begin with the disadvantages of competing methods (Baydin et al., 2018):

1. Symbolic differentiation. While it can in principle, give exact solutions as well as analytic answers to optimization problems, it can easily become computationally intractable due to *expression swell*: the expressions involved in calculating the derivative of an expression can grow exponentially.
2. Numerical differentiation. Finite difference schemes are simple to implement and only require values of the initial function. However, numerical approximations of derivatives are known to be ill-conditioned and unstable.

AD is a technique for evaluating derivatives, based on the chain rule, applied to functions implemented as computer programs. AD leverages the fact that any computer program, regardless of its complexity, computes its output by composing a finite set of elementary operations (binary operations together with unary transcendental functions such as $sin()$, $exp()$) defined by the programming language (Verma, 2000). The partial derivatives of these elementary functions are known. AD computes the overall derivatives of the program's output with respect to its inputs by systematically applying the chain rule to this composition of functions.

Note that not every computer program is differentiable: operations like abs(), round() or conditional statements (e.g., *if-else*) are non-differentiable or only differentiable at specific points. However, AD computes derivatives *at a given point*. A function is differentiable by AD at a point x if, when evaluated at x, every elementary operation in its computational graph is differentiable at the intermediate values it receives.

In this sense, any function that can be defined by the programming language (as a composition of differentiable elementary operations) is differentiable via AD. In particular, to perform backpropagation on a neural network, it is sufficient to implement the neural network in the programming language: the gradients can then be calculated via AD. In particular for PINNs, it suffices that the differential equation loss can be written in the language as a composition of differentiable elementary operations.

AD has two main modes of operation: forward mode and backward mode. Our main source for explaining them in the following sections is (Baydin et al., 2018).

1.2.1. Forward Mode AD

Forward accumulation, or tangent linear mode, operates by traversing the computational graph of a function $f : \mathbb{R}^n \to \mathbb{R}^m$ in its natural evaluation order. The method augments the standard computation (the *primal trace*) with a concurrent propagation of directional derivatives (the *tangent trace*).

For each intermediate variable v_i in the graph, its corresponding tangent $\dot{v}_i$ is computed to represent the directional derivative of v_i with respect to the input variables. The direction is defined by a "seed" vector $r \in \mathbb{R}^n$ with which the process is initialized by setting the input tangents $\dot{x} = r$. This is the vector in which direction we want to compute the gradient. The chain rule is then applied at each elementary operation to propagate these numerical derivative values forward through the graph.

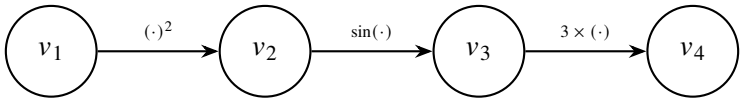
This process is illustrated in Figure 1.2 for the function $f(x) = 3\sin(x^2)$, evaluated at the point $x = 2$. To compute the derivative $\frac{df}{dx}$, the process is seeded with the tangent $\dot{x} = 1$.

The forward pass propagates numerical values for both the function and its derivative:

1. **Input:** The trace is initialized at $v_1 = x = 2$. The tangent is seeded with the chosen

Figure 1.2

The forward mode evaluation trace for $f(x) = 3\sin(x^2)$ at $x = 2$. The top row shows the dependency of intermediate variables. The Primal trace evaluates the function value, while the Tangent trace concurrently evaluates the derivative.



Primal:	$v_1 = x$ $= 2$	$v_2 = v_1^2$ $= 4$	$v_3 = \sin(v_2)$ ≈ -0.757	$v_4 = 3v_3$ ≈ -2.270
Tangent:	$\dot{v}_1 = 1$	$\dot{v}_2 = 2v_1\dot{v}_1$ $= 2(2)(1) =$ 4	$\dot{v}_3 = \cos(v_2)\dot{v}_2$ $\approx$ $-0.654(4) =$ -2.615	$\dot{v}_4 = 3\dot{v}_3$ $\approx$ $3(-2.615) =$ -7.845

direction, $\dot{v}_1 = 1$.

2. **Intermediate Steps:** At each subsequent operation, the chain rule is applied to the numerical results of the previous step. For $v_2 = v_1^2$, the primal value becomes 4, and its tangent is computed as $\dot{v}_2 = 2v_1\dot{v}_1 = 2(2)(1) = 4$. This process continues through the graph.

Upon completion of the forward pass, the final node contains both the function's output, $y = v_4 \approx -2.270$, and its derivative at the point $x = 2$, $\dot{y} = \dot{v}_4 \approx -7.845$.

More generally, a single forward pass of AD computes the Jacobian-vector product $J_f(x)r$. The full Jacobian matrix $J_f(x)$ can be recovered column-by-column through n forward passes, where each pass i is seeded with the canonical basis vector $r = e_i$ to compute the partial derivatives with respect to x_i.

The computational cost of a single forward pass is proportional to the cost of evaluating the function itself. Thus, for a function $f : \mathbb{R}^n \to \mathbb{R}$, obtaining the full gradient $\nabla f \in \mathbb{R}^n$ requires n passes. This cost becomes prohibitive for typical machine learning models, where n (the number of parameters) can be very large. This unfavorable scaling for high-dimensional inputs motivates the use of the alternative, backward mode of AD, for training neural networks.

1.2.2. Backward Mode AD

The computational inefficiency of forward mode for functions with high-dimensional inputs necessitates an alternative approach for machine learning applications. Backward mode accumulation, also known as *reverse mode AD*, solves this problem and is the foundational algorithm of backpropagation. This mode computes derivatives by traversing

the computational graph in reverse order.

Backward mode is a two-phase process. First, a standard forward pass is executed to compute the primal value for every intermediate variable v_i in the graph; these values are stored. The second phase is a reverse pass that propagates derivatives backward, starting from the final output.

In this mode, each intermediate variable v_i is associated with an *adjoint* variable $\bar{v}_i$, which represents the derivative of the final output y with respect to that intermediate variable:

$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

The backward pass is initiated by seeding the adjoint of the output variable, $\bar{y} = \frac{\partial y}{\partial y} = 1$. The chain rule is then applied in reverse to compute the adjoint of each variable based on the adjoints of the variables that depend on it. For a variable v_i that is used to compute a set of subsequent variables $\{v_j\}$, its adjoint is calculated by summing the contributions from all outgoing paths in the graph:

$$\bar{v}_i = \sum_{j \text{ where } v_j \text{ depends on } v_i} \bar{v}_j \frac{\partial v_j}{\partial v_i}$$

This process is illustrated for $f(x, y) = \sin(x)\cos(y)$ evaluated at the point $(x, y) = (\frac{\pi}{2}, \frac{\pi}{3})$ in Figure 1.3.

Figure 1.3
Computational trace for $f(x, y) = \sin(x)\cos(y)$ at $(x, y) = (\pi/2, \pi/3)$.

Step	Operation	Primal Value (v_i)	Adjoint ($\bar{v}_i$)	Adjoint Computation
v_1	Input x	$\pi/2$	0	$\bar{v}_1 = \bar{v}_3 \frac{\partial v_3}{\partial v_1} = \frac{1}{2} \cdot \cos(\frac{\pi}{2})$
v_2	Input y	$\pi/3$	$-\frac{\sqrt{3}}{2}$	$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = 1 \cdot (-\sin(\frac{\pi}{3}))$
v_3	$\sin(v_1)$	1	$\frac{1}{2}$	$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = 1 \cdot v_4$
v_4	$\cos(v_2)$	$\frac{1}{2}$	1	$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = 1 \cdot v_3$
v_5	$v_3 \times v_4$	$\frac{1}{2}$	1	(Seed)

The two-phase process unfolds as follows:

1. **Forward Primal Pass:** The function is evaluated, and all intermediate values are cached. This yields $v_1 = \frac{\pi}{2}$, $v_2 = \frac{\pi}{3}$, $v_3 = \sin(\frac{\pi}{2}) = 1$, $v_4 = \cos(\frac{\pi}{3}) = \frac{1}{2}$, and the final output $v_5 = 1 \cdot \frac{1}{2} = \frac{1}{2}$.

2. **Backward Adjoint Pass:** Derivatives are propagated backward through the computational trace, starting from the final variable and proceeding to the inputs.

- The pass is seeded at the output with an adjoint of one: $\bar{v}_5 = 1$.
- Adjoints for the inputs to the final multiplication are computed. The adjoint of v_4 is $\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 v_3 = 1 \cdot 1 = 1$. The adjoint of v_3 is $\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 v_4 = 1 \cdot \frac{1}{2} = \frac{1}{2}$.
- The adjoints are propagated further back. The adjoint for v_2 (which is y) is $\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = 1 \cdot (-\sin v_2) = -\sin(\frac{\pi}{3}) = -\frac{\sqrt{3}}{2}$.
- The adjoint for v_1 (which is x) is $\bar{v}_1 = \bar{v}_3 \frac{\partial v_3}{\partial v_1} = \frac{1}{2} \cdot (\cos v_1) = \frac{1}{2} \cdot \cos(\frac{\pi}{2}) = 0$.

In general, for a function $f : \mathbb{R}^n \to \mathbb{R}^m$, the computational cost of obtaining the full $m \times n$ Jacobian matrix depends on the mode of AD used. If we denote the operation count to evaluate the original function as $\text{ops}(f)$, the total cost for forward mode is $n \cdot c \cdot \text{ops}(f)$, as it requires one pass for each of the n input dimensions. Conversely, the cost for reverse mode is $m \cdot c \cdot \text{ops}(f)$, requiring one pass for each of the m output dimensions. The term c is a small constant factor, guaranteed to be less than 6 and typically in the range of 2 to 3 (Griewank and Walther, 2008).

This analysis reveals why reverse mode is the cornerstone of modern machine learning. The optimization of any neural network involves differentiating a scalar loss function with respect to a vast number of parameters ($m = 1, n \gg 1$), the exact scenario where reverse mode is most efficient.

In the context of PINNs, AD is not only used in the backpropagation step for training. It is used in evaluating the differential equation loss, $\mathcal{L}_{\mathcal{F}}(\theta)$, where the differential equation operator must be applied to the output function of the neural network, $\hat{u}(\theta)$. Here too, the input dimension (space, time) is usually larger than the output dimension.

2. FRACTIONAL DERIVATIVES AND PHYSICS-INFORMED METHODS

The standard Physics-Informed Neural Network framework, while powerful, is fundamentally designed around integer-order differential operators. Extending this paradigm to Fractional Differential Equations (FDEs) requires confronting the core challenge of fractional calculus: the non-local nature of its operators. This chapter provides a self-contained exposition of this problem and the methods developed to address it.

We begin by formally introducing the classical fractional derivatives and explaining why their integral-based definitions are incompatible with the chain-rule mechanism of Automatic Differentiation (AD). We then provide a detailed review of the state-of-the-art methodology, the Fractional Physics-Informed Neural Network (fPINN), explaining its hybrid structure and its inherent trade-offs. Finally, we introduce the conformable derivative as an alternative, AD-native operator and present a practical framework for its implementation, establishing the central theory that motivates the conformable PINN approach and numerical investigations in this thesis.

2.1. The Challenge of Non-Locality in Deep Learning

Classical calculus, founded upon integer-order derivatives, models systems where change is local and instantaneous. However, a vast array of phenomena in science and engineering exhibit non-local behaviors, where the future state of a system depends on its entire history. Such systems, including viscoelastic materials and anomalous diffusion processes, are said to possess "memory" or hereditary properties and are modeled using fractional calculus (Podlubny, 1998).

The cornerstone of this field is the *fractional derivative*, an operator defined by a non-local integral. Two of the most canonical definitions are the Riemann-Liouville and the Caputo derivatives (Diethelm, 2010).

Definition 2.1.1 (Riemann-Liouville Fractional Derivative)**.** *The Riemann-Liouville fractional derivative of order $\alpha \in [n-1, n)$ for a function $f(t)$ is defined as:*

$$ {}_{a}^{RL}D_{t}^{\alpha} f(t) = \frac{1}{\Gamma(n-\alpha)} \frac{d^n}{dt^n} \int_a^t \frac{f(\tau)}{(t-\tau)^{\alpha-n+1}} d\tau \tag{2.1} $$

Definition 2.1.2 (Caputo Fractional Derivative)**.** *The Caputo fractional derivative of order $\alpha \in [n-1, n)$ for a function $f(t)$ is defined as:*

$$ {}_{a}^{C}D_{t}^{\alpha} f(t) = \frac{1}{\Gamma(n-\alpha)} \int_a^t \frac{f^{(n)}(\tau)}{(t-\tau)^{\alpha-n+1}} d\tau \tag{2.2} $$

Regardless of the specific definition, the integral formulation allows Fractional Differential Equations (FDEs) to capture the historical dependencies inherent in memory-effects. However, this non-local, integral-based nature creates a fundamental conflict with the core mechanism of modern deep learning: Automatic Differentiation (AD). The efficacy of AD is predicated on the local nature of the chain rule. A true non-local operator does not fit this computational structure, as the standard chain rule is not valid in fractional calculus and often takes the form of an infinite series (Pang et al., 2019; Tarasov, 2016). This mathematical and computational incompatibility necessitates specialized frameworks for solving FDEs with neural networks.

2.2. fPINNs: The State-of-the-Art Methodology

The established method for applying neural networks to FDEs is the Fractional Physics-Informed Neural Network (fPINN), introduced by Pang et al., 2019. This framework modifies the standard PINN architecture to accommodate the non-local fractional operators by employing a hybrid approach in the loss function.

2.2.1. The Hybrid Loss Function

The key innovation of the fPINN framework is a hybrid approach for constructing the physics-informed loss function. The central idea is to decompose a general fractional operator, $\mathcal{L}$, into two components: an AD-compatible part, $\mathcal{L}_{AD}$, containing integer-order derivatives, and a non-AD-compatible part, $\mathcal{L}_{nonAD}$, containing the non-local fractional operators. The framework then approximates the non-local term with a numerical discretization scheme, denoted $\mathcal{L}_{FDM}$.

Definition 2.2.1 (fPINN Hybrid Loss Function)**.** *Given a fractional PDE $\mathcal{L}\{\boldsymbol{u}(\mathbf{x}, t)\} = \boldsymbol{f}(\mathbf{x}, t)$, where the operator is decomposed as $\mathcal{L} = \mathcal{L}_{AD} + \mathcal{L}_{nonAD}$, the hybrid physics loss is defined as:*

$$\mathcal{L}_{\mathcal{F}}(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} \|\mathcal{L}_{FDM}(\hat{\boldsymbol{u}}_\theta(z_f^i)) + \mathcal{L}_{AD}(\hat{\boldsymbol{u}}_\theta(z_f^i)) - \boldsymbol{f}(z_f^i)\|^2 \tag{2.3}$$

The term corresponding to $\mathcal{L}_{AD}$ is computed using standard AD. The non-local operators in $\mathcal{L}_{nonAD}$, such as the Directional Fractional Laplacian, are approximated using a numerical scheme to construct the $\mathcal{L}_{FDM}$ term. For instance, to approximate the directional derivatives required by the fractional Laplacian, the fPINN methodology employs well-established numerical schemes such as the Grünwald-Letnikov (GL) formula. This numerical approximation is then embedded directly into the loss function, enabling the training of the neural network.

2.2.2. Advantages and Limitations of the fPINN Approach

The fPINN framework is a powerful and versatile methodology for solving FDEs.

- **Advantages:** Like standard PINNs, fPINNs are mesh-free and well-suited for complex geometries. They are particularly effective for problems with sparse or noisy data, as they can directly evaluate the loss at measurement points without requiring interpolation, a significant source of error for traditional methods (Pang et al., 2019).
- **Limitations:** The primary trade-off of the fPINN methodology is its reliance on numerical discretization. This introduces two key limitations. First, it embeds a *discretization error* from the finite difference scheme directly into the training process, which can lead to error saturation. Second, the reliance on grid-based formulas like FDM for the non-local part can suffer from the *curse of dimensionality*, making the approach computationally expensive for problems in high dimensions.

2.3. Conformable Derivatives: An AD-Native Alternative

An alternative approach to the fPINN methodology is to utilize a class of operators that are local by definition and thus inherently compatible with the chain-rule mechanism of Automatic Differentiation. This motivates the use of the *conformable derivative*. The original work by Katugampola, 2014 introduced an alternative definition for the fractional derivative designed to retain classical properties, with further analysis and properties explored in works such as Abdeljawad, 2015.

2.3.1. The Conformable Derivative

Unlike the classical Riemann-Liouville and Caputo derivatives, the conformable derivative was designed to satisfy the standard properties of integer-order calculus, making it analytically more tractable.

Definition 2.3.1 (Conformable Derivative (Katugampola, 2014))**.** *For a function $f : [a, \infty) \to \mathbb{R}$ and order $\alpha \in (0, 1]$, the conformable derivative is defined via the limit:*

$$T_\alpha(f)(t) = \lim_{\varepsilon \to 0} \frac{f(t + \varepsilon t^{1-\alpha}) - f(t)}{\varepsilon} \tag{2.4}$$

This definition intentionally mirrors the standard limit definition of the first derivative. As a result, it inherits many of the most important properties of classical calculus, including linearity, the product rule, the quotient rule, and crucially for our purposes, the chain rule (Abdeljawad, 2015). For functions that are once-differentiable, it simplifies to a convenient rescaling of the classical derivative: $T_\alpha(f)(t) = t^{1-\alpha} f'(t)$.

However, while this operator is mathematically elegant and fully compatible with AD, its strictly local nature is a significant drawback for physical modeling. The derivative at

a point t depends only on the values of the function in an infinitesimal neighborhood of t. It does not incorporate the function's history from the starting point a, and therefore fails to capture the "memory" effects that are the primary motivation for using fractional calculus in many applications.

2.3.2. The Improved Conformable Derivative

Recognizing the limitations of the local definition, an "improved" conformable derivative was proposed by Gao and Chi, 2020. This formulation was specifically designed to serve as a better approximation of the non-local Caputo and Riemann-Liouville operators while preserving the computational advantages of locality.

The structure of the improved derivative is a convex combination of a global "memory" term and a local "derivative" term. The parameter α controls the balance between these two components.

Definition 2.3.2 (Improved Conformable Derivatives for $\alpha \in (0, 1]$)**.** *For a function $f : [a, \infty) \to \mathbb{R}$, the improved conformable derivatives are defined by the limits:*

1. *The Caputo-type derivative:*

$$ {}_a^C\tilde{T}_\alpha(f)(t) = \lim_{\varepsilon \to 0} \left[(1-\alpha)(f(t) - f(a)) + \alpha \frac{f(t + \varepsilon(t-a)^{1-\alpha}) - f(t)}{\varepsilon} \right] \tag{2.5} $$

2. *The Riemann-Liouville-type derivative:*

$$ {}_a^{RL}\tilde{T}_\alpha(f)(t) = \lim_{\varepsilon \to 0} \left[(1-\alpha) f(t) + \alpha \frac{f(t + \varepsilon(t-a)^{1-\alpha}) - f(t)}{\varepsilon} \right] \tag{2.6} $$

These definitions can be generalized to higher orders of α. For a fractional order $\alpha \in (n, n+1]$ where $n \in \mathbb{N}$, the definitions incorporate the n-th order classical derivative.

Definition 2.3.3 (Generalized Improved Conformable Derivatives for $\alpha \in (n, n+1]$)**.** *For a function f that is $(n+1)$-times differentiable, the generalized improved conformable derivatives are:*

1. *The Caputo-type derivative:*

$$ \begin{aligned} {}_a^C\tilde{T}_\alpha(f)(t) = \lim_{\varepsilon \to 0} \Bigg[& (n+1-\alpha) \left(f^{(n)}(t) - f^{(n)}(a) \right) \\ & + (\alpha - n) \frac{f^{(n)}(t + \varepsilon(t-a)^{n+1-\alpha}) - f^{(n)}(t)}{\varepsilon} \Bigg] \end{aligned} \tag{2.7} $$

2. *The Riemann-Liouville-type derivative:*

$$ {}_a^{RL}\tilde{T}_\alpha(f)(t) = \lim_{\varepsilon \to 0} \left[(n+1-\alpha) f^{(n)}(t) + (\alpha - n) \frac{f^{(n)}(t + \varepsilon(t-a)^{n+1-\alpha}) - f^{(n)}(t)}{\varepsilon} \right] \tag{2.8} $$

For the practical case of sufficiently differentiable functions, such as the neural networks used in this work, these limit definitions simplify to convenient explicit formulas that are ideal for implementation.

Proposition 2.3.1 (Explicit Formulas for Improved Conformable Derivatives)**.** *If a function f is $(n+1)$-times differentiable and $\alpha \in (n, n+1]$, the explicit formulas are:*

$$ {}_{a}^{RL}\tilde{T}_{\alpha}(f)(t) = (n+1-\alpha)f^{(n)}(t) + (\alpha - n)(t-a)^{n+1-\alpha} f^{(n+1)}(t) \tag{2.9} $$

$$ {}_{a}^{C}\tilde{T}_{\alpha}(f)(t) = (n+1-\alpha)(f^{(n)}(t) - f^{(n)}(a)) + (\alpha - n)(t-a)^{n+1-\alpha} f^{(n+1)}(t) \tag{2.10} $$

These improved operators offer a crucial compromise: they remain computationally local, but they incorporate historical information (e.g., the term involving $f(a)$ or $f^{(n)}(a)$) to provide a superior approximation to the classical non-local operators. Their explicit reliance on integer-order derivatives provides a direct pathway for implementation within an AD framework.

3. METHODOLOGY

This chapter details the methodology and the computational framework used to implement the conformable derivative in a PINN, a methodology we term the Conformable Fractional Physics-Informed Neural Network (cfPINN). We begin by formally defining the cfPINN framework, focusing on the concept of the surrogate model and the resulting trade-off between discretization error and modeling error. We then discuss the application of this framework to both forward and inverse problems.

The second part of the chapter is dedicated to the practical implementation of the cfPINN. We describe the software stack, the architecture of the neural network surrogates, and the specific, AD-native implementation of the one- and multi-dimensional conformable operators that form the core of our method.

The central idea of the cfPINN is to replace the numerically-discretized non-local operator found in the fPINN loss function with a local, fully-differentiable conformable operator. This avoids the introduction of discretization error but fundamentally changes the problem being solved.

3.1. Surrogate Model Architecture and Modeling Error

Consider a true physical process described by a non-local Fractional Differential Equation (FDE) with a classical Caputo operator of order α:

$$\mathcal{F}(u(z), {}^{C}D_{a}^{\alpha}u(z), \ldots; \gamma) = f(z), \quad z \in \Omega \tag{3.1}$$

$$\mathcal{B}(u(z)) = g(z), \quad z \in \partial\Omega \tag{3.2}$$

The cfPINN framework does not solve this equation directly. Instead, it solves a related *surrogate problem* where the non-local operator is replaced by its improved conformable counterpart of order $\hat{\alpha}$:

$$\mathcal{F}(u(z), {}_{a}^{C}\tilde{T}_{\hat{\alpha}}u(z), \ldots; \gamma) = f(z), \quad z \in \Omega \tag{3.3}$$

$$\mathcal{B}(u(z)) = g(z), \quad z \in \partial\Omega \tag{3.4}$$

The neural network uses Automatic Differentiation to find a solution that satisfies this local, conformable FDE to a high degree of precision. The difference between the solution of the true non-local problem and the solution of this surrogate problem is the *surrogate modeling error*. As noted by Gao and Chi, 2020, the improved conformable derivative is a good approximation of the classical fractional derivative, but it is not identical. Therefore, we do not expect the optimal effective order $\hat{\alpha}$ of the surrogate model to be equal to the true physical order α of the non-local system.

This defines the fundamental trade-off of our approach: we have eliminated the *discretization error* and high-dimensional scaling limitations of the fPINN framework, but at the cost of introducing an inherent *surrogate modeling error*.

3.2. Forward problems

When applying the cfPINN to forward problems, where the goal is to find the solution $u(t)$ for a known fractional order α, the approach depends on whether sparse measurements of the true system are available.

Scenario 1: Fixed Fractional Order (No Data) If no observational data is available, we must assume the order of the surrogate model is equal to the known physical order, $\hat{\alpha} = \alpha$. The network is then trained to minimize the physics residual of the surrogate problem. The loss function depends only on the network parameters θ:

$$\mathcal{L}(\theta) = \mathcal{L}_{phys}(\theta) = \frac{1}{N_{col}} \sum_{i=1}^{N_{col}} \|\mathcal{R}(z_i^c)\|^2, \tag{3.5}$$

where the physics residual $\mathcal{R}(z)$ is defined by the surrogate FDE with a fixed order α:

$$\mathcal{R}(z) := \mathcal{F}(\hat{u}(z), {}_a^C\tilde{T}_\alpha \hat{u}(z), \dots) - f(z). \tag{3.6}$$

The optimization problem is a standard search for the optimal network weights:

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta). \tag{3.7}$$

In the context of this work, we always use the Adam optimizer (Kingma & Ba, 2014) where the learning rate η will be specified in each problem. This optimizer is also the choice in the original fPINN work (Pang et al., 2019). In this case, the accuracy of the final solution is almost entirely limited by the magnitude of the surrogate modeling error, which can be significant, as demonstrated by Gao and Chi, 2020 and observed in our own simulations in Chapter 4.

Scenario 2: Trainable Effective Order with Sparse Data If a sparse set of measurements from the true system is available, we can treat the order of the surrogate model, $\hat{\alpha}$, as a trainable parameter to be learned from the data. This reframes the task as a joint optimization problem over both the network weights θ and the effective fractional order $\hat{\alpha}$.

Definition 3.2.1 (Joint Loss Function for Forward Problems with Data)**.** *The composite loss function depends on both the network parameters θ and the effective fractional order $\hat{\alpha}$:*

$$\mathcal{L}(\theta, \hat{\alpha}) = \mathcal{L}_{phys}(\theta, \hat{\alpha}) + \lambda \mathcal{L}_{data}(\theta) \tag{3.8}$$

where the components are defined as follows:

- *The physics loss, $\mathcal{L}_{phys}$, is the mean squared error of the physics residual of the surrogate FDE, which is now a function of the trainable order $\hat{\alpha}$:*

$$\mathcal{L}_{phys}(\theta, \hat{\alpha}) = \frac{1}{N_{col}} \sum_{i=1}^{N_{col}} \|\mathcal{F}(\hat{u}(z_i^c), {}_a^C\tilde{T}_{\hat{\alpha}}\hat{u}(z_i^c), \dots) - f(z_i^c)\|^2. \tag{3.9}$$

- *The data loss, $\mathcal{L}_{data}$, remains the mean squared error between the network's prediction and the N_{data} measured data points.*

The goal is to find the optimal pair $(\theta^*, \hat{\alpha}^*)$ that jointly minimizes this loss function. This is expressed as:

$$(\theta^*, \hat{\alpha}^*) = \arg\min_{\theta, \hat{\alpha}} \mathcal{L}(\theta, \hat{\alpha}). \tag{3.10}$$

In this scenario, the network learns an effective order $\hat{\alpha} \neq \alpha$ that allows the local model to best fit the non-local data. As we will demonstrate in Chapter 4, this approach can yield surprisingly accurate results by mitigating the surrogate modeling error.

Inverse Problems For inverse problems, where the primary goal is to discover the true physical order α from data, the cfPINN approach is challenging. The surrogate modeling error makes it difficult to disentangle the true physical parameter α from the effective parameter $\hat{\alpha}$ that best fits the data. The learned parameter $\hat{\alpha}$ should be interpreted as an effective order for the surrogate model, not necessarily the true order of the underlying non-local system.

A significant advantage of the cfPINN framework is its direct reliance on the analytical form of the conformable derivative. Any future theoretical improvements in conformable derivative formulas that better approximate their non-local counterparts will directly translate into a reduction of the surrogate modeling error and a direct improvement of this method.

3.3. Implementation in JAX

Our framework is implemented in Python using the JAX library (Bradbury et al., 2018). The complete code for both the implementation and the numerical experiments in Chapter 4, is available in a public GitHub repository[3].

3.3.1. One-Dimensional Operators

For one-dimensional problems, we work with the improved Caputo-type conformable derivative unless otherwise stated, due to its convenient handling of initial conditions. The implementation directly translates its explicit formula for once-differentiable functions into JAX code.

[3] https://github.com/davidspn/cfpinns

Definition 3.3.1 (1D Improved Caputo-Type Conformable Derivative)**.** *For a once-differentiable function* $f : [a, \infty) \to \mathbb{R}$ *and order* $\alpha \in (0, 1]$*, the explicit formula is:*

$$ {}_a^C\tilde{T}_\alpha(f)(t) = (1-\alpha)(f(t) - f(a)) + \alpha(t-a)^{1-\alpha} f'(t) \tag{3.11} $$

Because the fractional order α appears directly as a variable in this equation, the physics loss $\mathcal{L}_{phys}$ becomes a differentiable function of α. Consequently, JAX can automatically compute the gradient of the total loss with respect to α just as it does for the network weights, making it a learnable parameter with no extra effort. A simplified code implementation is shown in Listing 3.1.

```python
import jax
import jax.numpy as jnp
from typing import Callable

_get_first_derivative = jax.grad

def compute_caputo_0_to_1(
    f: Callable,
    t: jnp.ndarray,
    a: float,
    alpha: jnp.ndarray
) -> jnp.ndarray:

    f_prime = _get_first_derivative(f)

    f_at_t = jax.vmap(f)(t)
    f_at_a = f(a)
    f_prime_at_t = jax.vmap(f_prime)(t)

    f_diff = f_at_t - f_at_a
    coord_diff = t - a

    term1 = (1.0 - alpha) * f_diff
    term2 = alpha * jnp.power(coord_diff + 1e-9, 1.0 - alpha) *
   f_prime_at_t

    return term1 + term2
```

CÓDIGO 3.1. A JAX function for computing the improved conformable derivative for $\alpha \in (0, 1)$.

Among other important 1D operators is the 1D fractional Laplacian, which is widely used in modeling anomalous diffusion and other non-local processes (Metzler & Klafter, 2000).

The 1D non-local fractional Laplacian is defined as a sum of left- and right-sided derivatives. Our conformable approximation is constructed naturally from this definition:

Definition 3.3.2 (1D Fractional Laplacian)**.** *The one-dimensional fractional Laplacian is*

defined as:

$$(-\Delta)^{\alpha/2}u(x) := \frac{1}{2\cos(\pi\alpha/2)}\left(D^{\alpha}_{0+}u(x) + D^{\alpha}_{1-}u(x)\right) \tag{3.12}$$

where D^{α}_{0+} and D^{α}_{1-} are the left- and right-sided Riemann-Liouville fractional derivatives.

Definition 3.3.3 (Conformable Approximation of the 1D Fractional Laplacian)**.** *Our approximate local fractional Laplacian mirrors the structure of the non-local operator:*

$$-(\tilde{\Delta})^{\hat{\alpha}/2}u(x) := \frac{1}{2\cos(\pi\hat{\alpha}/2)}\left(({}^{C}_{0}\tilde{T}_{\hat{\alpha}}u)(x) + ({}^{C}_{0}\tilde{T}_{\hat{\alpha}}\tilde{u})(1-x)\right) \tag{3.13}$$

where the right-sided term is the left-sided derivative of the reflected function $\tilde{u}(x) = u(1-x)$.

3.3.2. Multidimensional Problems

For multi-dimensional problems, we construct a surrogate for the fractional Laplacian based on its directional representation.

Definition 3.3.4 (Directional Representation of the Fractional Laplacian)**.** *The fractional Laplacian is characterized by its directional representation (Lischke et al., 2019):*

$$(-\Delta)^{\alpha/2}u(\mathbf{x}) = C_{\alpha,d}\int_{|\boldsymbol{\theta}|=1} D^{\alpha}_{\boldsymbol{\theta}}u(\mathbf{x})d\boldsymbol{\theta} \tag{3.14}$$

where $D^{\alpha}_{\boldsymbol{\theta}}$ is the Riemann-Liouville fractional directional derivative.

Our framework replaces the non-local directional derivative $D^{\alpha}_{\boldsymbol{\theta}}$ inside the integral with its one-dimensional improved conformable counterpart, ${}^{RL}_{0}\tilde{T}_{\hat{\alpha}}$.

Definition 3.3.5 (Conformable Approximation of the Directional Laplacian)**.** *Our local surrogate for the fractional Laplacian is defined as:*

$$-(\tilde{\Delta})^{\hat{\alpha}/2}u(\mathbf{x}) := C_{\hat{\alpha},d}\int_{|\boldsymbol{\theta}|=1}\left({}^{RL}_{0}\tilde{T}_{\hat{\alpha}}u_{\boldsymbol{\theta}}\right)(s)d\boldsymbol{\theta} \tag{3.15}$$

where $u_{\boldsymbol{\theta}}(s)$ is a 1D slice of the function $u(\mathbf{x})$ along the direction $\boldsymbol{\theta}$.

To evaluate the integral over the unit sphere, we use a standard Monte Carlo method, averaging the value of the 1D conformable operator over a number of random directions $\boldsymbol{\theta}$ sampled uniformly at each collocation point. In contrast, fPINN implementations often use numerical quadrature rules, such as Gauss-Legendre quadrature, to evaluate their discretized operators (Pang et al., 2019). Our Monte Carlo approach is distinct from recent works like Guo et al., 2022, where Monte Carlo methods are used to approximate the non-local fractional operator itself; here, we only use Monte Carlo to approximate the outer integral, while the inner conformable operator is evaluated exactly via AD.

An alternative approach is to define a surrogate model for the Laplacian operator directly. The study of Laplace's equation within the conformable derivative framework is

a relatively new area, with researchers noting that it has not been extensively investigated (Martínez et al., 2021). Inspired by the structure of the classical Laplacian, a *conformable fractional Laplacian* can be constructed by applying the 1D conformable operator of order $\alpha/2$ twice along each spatial dimension:

Definition 3.3.6 (Conformable Fractional Laplacian)**.** *A possible conformable surrogate for the 2D fractional Laplacian of order α is given by:*

$$(\tilde{\Delta})^{\alpha} u(x, y) := T_{\alpha/2,x}(T_{\alpha/2,x}(u(x, y))) + T_{\alpha/2,y}(T_{\alpha/2,y}(u(x, y))), \tag{3.16}$$

where $T_{\alpha/2}$ is the (not improved) conformable derivative of order $\alpha/2$.

We investigated this particular formulation as a natural extension. However, preliminary experiments with this approach were not promising, as the surrogate modeling error was significantly larger than that of the directional approximation. To our knowledge, an improved multi-dimensional conformable laplace operator has not yet been proposed in the literature.

4. NUMERICAL SIMULATIONS

This chapter presents a series of numerical experiments designed to validate the conformable fractional PINN (cfPINN) framework and investigate its properties. We address several benchmark problems from the fractional calculus literature, covering forward problem formulations both with fixed parameters and with effective parameters learned from sparse data. For each experiment, we provide a formal problem statement, a detailed summary of the hyperparameters used, and a discussion of the results.

4.1. One-Dimensional Fractional Initial Value Problem

We begin with a benchmark problem from Gao and Chi, 2020, which is valuable for its known analytical solutions under several different definitions of the fractional derivative. This allows for a clear analysis of the surrogate modeling error.

4.1.1. Problem Statement

We consider a one-dimensional fractional initial value problem (IVP). The true non-local problem is governed by the classical Caputo derivative of order $\alpha = 0.5$:

$$ {}^{C}D_{0}^{0.5}y(x) = \frac{2}{\Gamma(2.5)}x^{3/2}, \quad y(0) = 0. \tag{4.1} $$

The local surrogate problem, which our cfPINN solves, is governed by the improved conformable derivative of order $\hat{\alpha}$:

$$ {}_{0}^{C}\tilde{T}_{\hat{\alpha}}y(x) = \frac{2}{\Gamma(2.5)}x^{3/2}, \quad y(0) = 0. \tag{4.2} $$

The known analytical solutions for the true non-local problem and for the surrogate problem with different derivative definitions are:

- *Non-local Caputo solution:* $y(x) = x^2$.
- *Standard Conformable solution:* $y(x) = \frac{1}{\Gamma(2.5)}x^2$.
- *Improved Conformable solution:* $y(x) = \frac{4}{\Gamma(2.5)}\left(x^{3/2} - \frac{3}{2}x + \frac{3}{2}\sqrt{x} - \frac{3}{4}\right) + \frac{3}{\Gamma(2.5)}e^{-2\sqrt{x}}$.

Our objective is to investigate two forward problem scenarios: one with a fixed fractional order, and one where an effective order is learned from sparse data to improve the solution accuracy.

4.1.2. Methodology and Hyperparameters

The cfPINN is constructed as described in Chapter 3. The initial condition $y(0) = 0$ is enforced via the ansatz $\hat{y}(x) = x \cdot NN(x; \theta)$. For the data-assisted case, $\hat{\alpha}$ is a trainable parameter constrained to $(0, 1)$ via a sigmoid transformation. The hyperparameters are detailed in Table 4.1.

Parameter	**Forward Problem (Fixed α)**	**Forward Problem (Learnable $\hat{\alpha}$)**
Neural Network Architecture	2 hidden layers, 32 neurons/layer	2 hidden layers, 32 neurons/layer
Optimizer	Adam	Adam
Learning Rate	1×10^{-3}	1×10^{-3}
Training Epochs	20,000	20,000
Collocation Points (N_{col})	100	100
Data Points (N_{data})	0	5
Data Loss Weight (λ)	N/A	100
Fractional Order	Fixed $\alpha = 0.5$	Learned $\hat{\alpha} \in (0, 1)$

Table 4.1
Hyperparameters for the 1D Fractional IVP experiments.

4.1.3. Results and Discussion

Forward Problem with Fixed Order

In this scenario, the network successfully converges to the known analytical solution of the improved conformable IVP. Figure 4.1 compares this cfPINN solution to the other analytical solutions, confirming that the improved conformable operator provides a better approximation to the non-local operator than the standard one, though a clear error remains. The convergence of the physics loss and L2 error is shown in Figure 4.2 and Figure 4.3, respectively. The final L2 relative error with respect to the improved conformable analytical solution is on the order of 10^{-4}. This shows the approach is highly effective in learning the conformable solution, which is expected as it reduces to a standard PINN problem that is compatible with AD. Thus, most of the error with respect to the non-local problem is due to the surrogate modeling error.

Forward Problem with a Learnable Effective Order

In this scenario, the network learns an effective fractional order $\hat{\alpha}$ by fitting the sparse data from the true non-local solution. Figure 4.4 shows that the learned parameter converges to a value that is distinct from the true non-local order of $\alpha = 0.5$. As the optimizer finds this effective order, the L2 relative error of the cfPINN solution with respect to the true non-local solution decreases, as shown in the trajectory plot in Figure 4.5. The final

cfPINN solution, obtained using the learned $\hat{\alpha}$, provides a surprisingly accurate match to the true non-local solution (Figure 4.6), with a final L2 relative error on the order of 10^{-2}. While this demonstrates the effectiveness of learning an effective order, it is noteworthy that state-of-the-art fPINN methods, which directly discretize the non-local operator, can achieve higher accuracy, typically around 10^{-4} for similar problems (Pang et al., 2019).

Figure 4.1

Forward Problem (Fixed α): Comparison of the cfPINN solution to the exact analytical solutions.

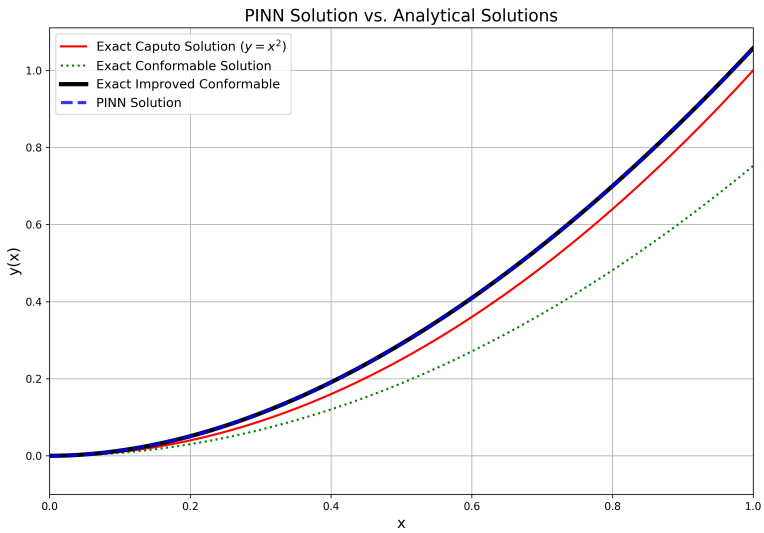


Figure 4.2

Forward Problem (Fixed α): Training loss over iterations.

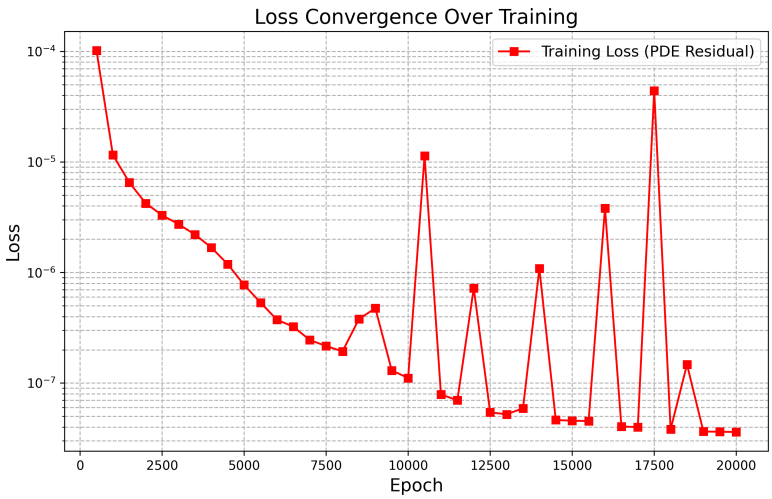


Figure 4.3

Forward Problem (Fixed α): L2 error relative to the improved conformable solution over training iterations.

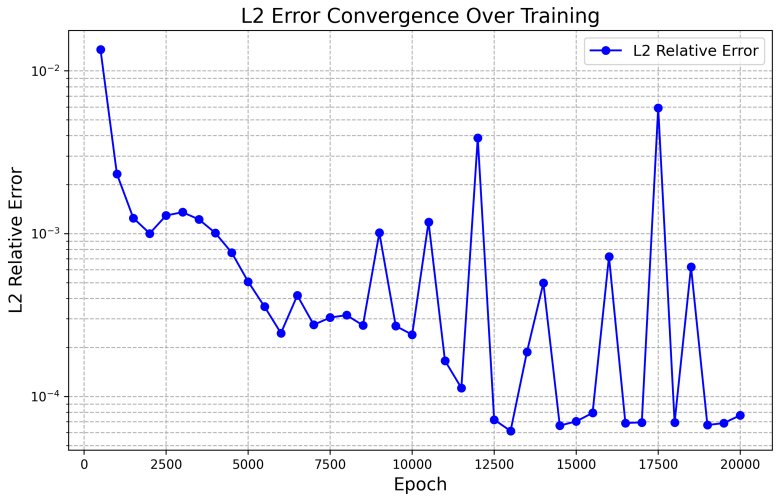


Figure 4.4

Forward Problem (Learnable $\hat{\alpha}$): Convergence of the learned effective fractional order.

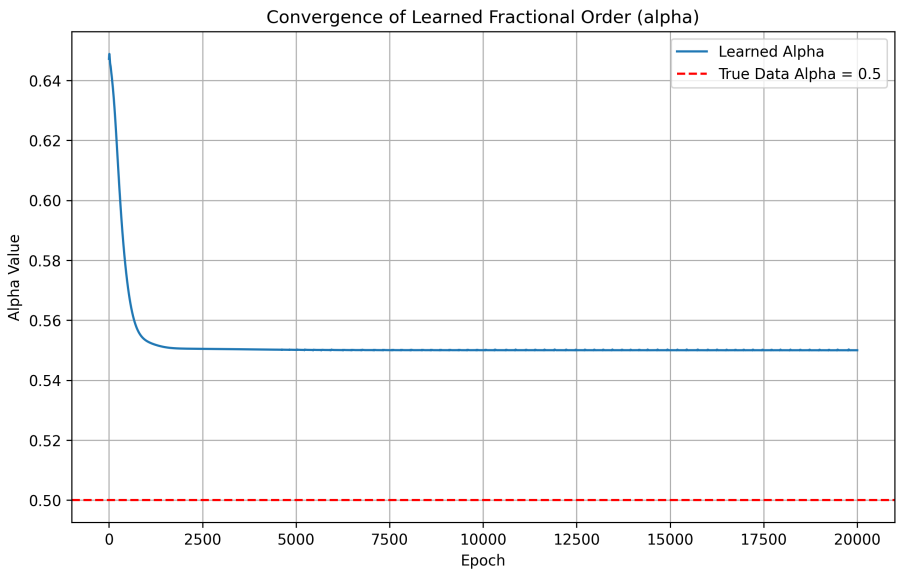


Figure 4.5

Forward Problem (Learnable $\hat{\alpha}$): The L2 relative error of the cfPINN solution versus the learned effective order during the training trajectory.

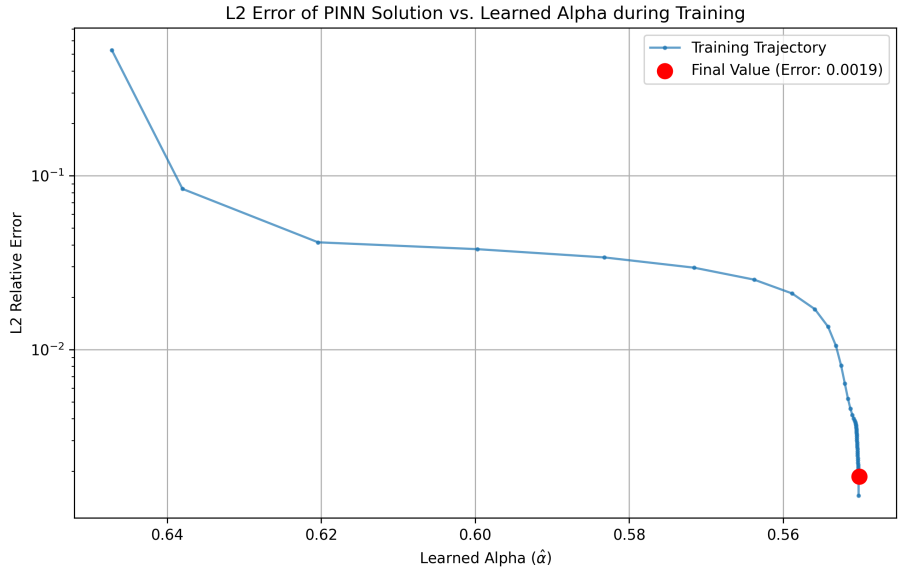
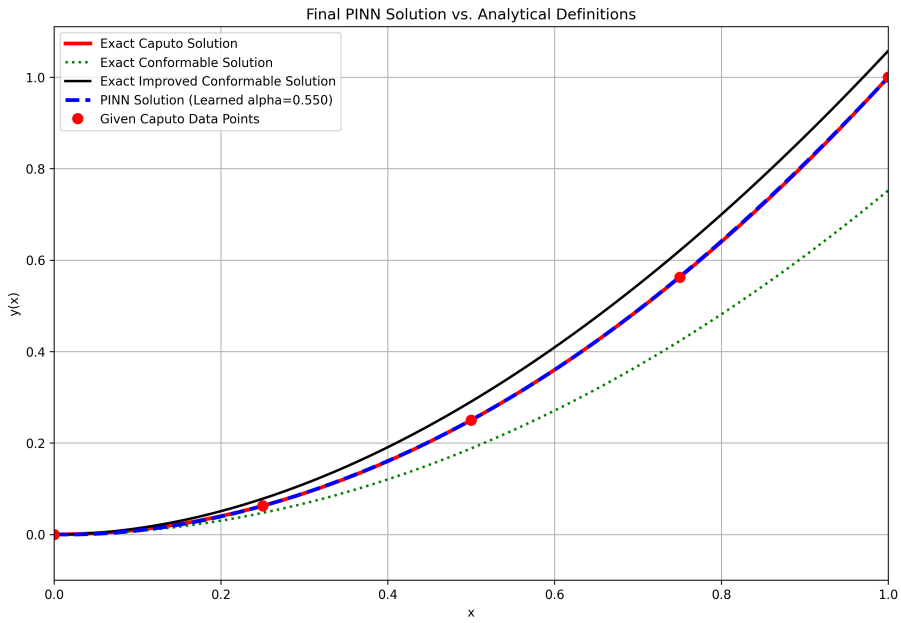


Figure 4.6

Forward Problem (Learnable $\hat{\alpha}$): Comparison of the final cfPINN solution against the analytical solutions. The model accurately matches the true non-local Caputo solution.



4.2. One-Dimensional Fractional Poisson Problem

To further test the framework, we address a standard benchmark from the fPINN literature: the one-dimensional fractional Poisson problem (Pang et al., 2019).

4.2.1. Problem Statement

The true non-local problem is defined by the fractional Laplacian operator:

$$(-\Delta)^{\alpha/2}u(x) = f(x), \quad x \in (0,1) \tag{4.3}$$

subject to homogeneous boundary conditions $u(0) = u(1) = 0$. For this problem, a fabricated smooth solution is considered (Pang et al., 2019):

$$u(x) = x^3(1-x)^3 \tag{4.4}$$

The corresponding forcing term $f(x)$ is derived analytically by applying the fractional Laplacian operator to this known solution with a ground-truth order of $\alpha = 1.5$, resulting in:

$$f(x) = \frac{1}{2\cos(\pi\alpha/2)}\left[\frac{\Gamma(4)}{\Gamma(4-\alpha)}\left(x^{3-\alpha} + (1-x)^{3-\alpha}\right) - \frac{3\Gamma(5)}{\Gamma(5-\alpha)}\left(x^{4-\alpha} + (1-x)^{4-\alpha}\right) + \frac{3\Gamma(6)}{\Gamma(6-\alpha)}\left(x^{5-\alpha} + (1-x)^{5-\alpha}\right) - \frac{\Gamma(7)}{\Gamma(7-\alpha)}\left(x^{6-\alpha} + (1-x)^{6-\alpha}\right)\right]$$

Our objective is to solve this forward problem by leveraging sparse data to learn an effective fractional order $\hat{\alpha}$ for the surrogate model.

4.2.2. Methodology and Hyperparameters

The boundary conditions are enforced via the ansatz $\hat{u}(x) = x(1-x)\cdot NN(x;\theta)$. To ensure the problem is well-posed, the forcing term $f(x)$ in the physics residual is held constant. The learnable parameter $\hat{\alpha}$ is constrained to the interval $(1,2)$. The hyperparameters are summarized in Table 4.2.

Parameter	**Value**
Neural Network Architecture	2 hidden layers, 64 neurons/layer
Optimizer	Adam
Learning Rate	1×10^{-4}
Training Epochs	100,000
Collocation Points (N_{col})	500
Data Points (N_{data})	10
Data Loss Weight (λ)	1000
Ground-Truth Order (α)	1.5
Learned Parameter	Effective order $\hat{\alpha} \in (1,2)$

Table 4.2
Hyperparameters for the 1D Fractional Poisson problem with a learnable order.

4.2.3. Results and Discussion

The training dynamics are shown in Figure 4.7 and Figure 4.8. The cfPINN successfully discovers an approximate fractional order and produces a solution that accurately matches the analytic non-local solution, as shown in Figure 4.9. The final L2 relative error of the solution is on the order of 10^{-1}. This result, while demonstrating a successful parameterization of the forward problem, is less accurate than the errors reported for fPINN methods on similar problems, which are often in the range of 10^{-4} (Pang et al., 2019), highlighting the impact of the surrogate modeling error.

Figure 4.7

Training dynamics for the Poisson problem: Convergence of the learned approximate order $\hat{\alpha}$.

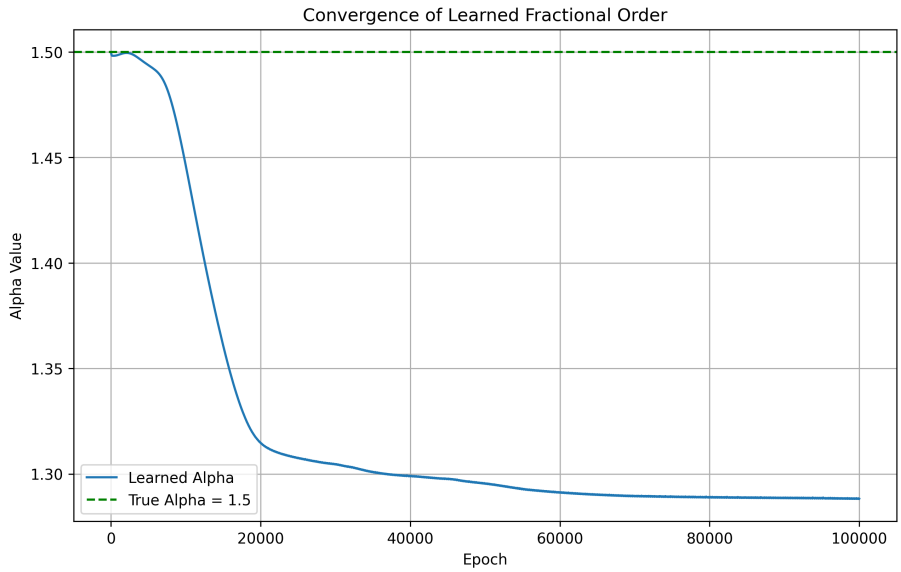


Figure 4.8

Training dynamics for the Poisson problem: L2 error vs. the learned approximate order $\hat{\alpha}$.

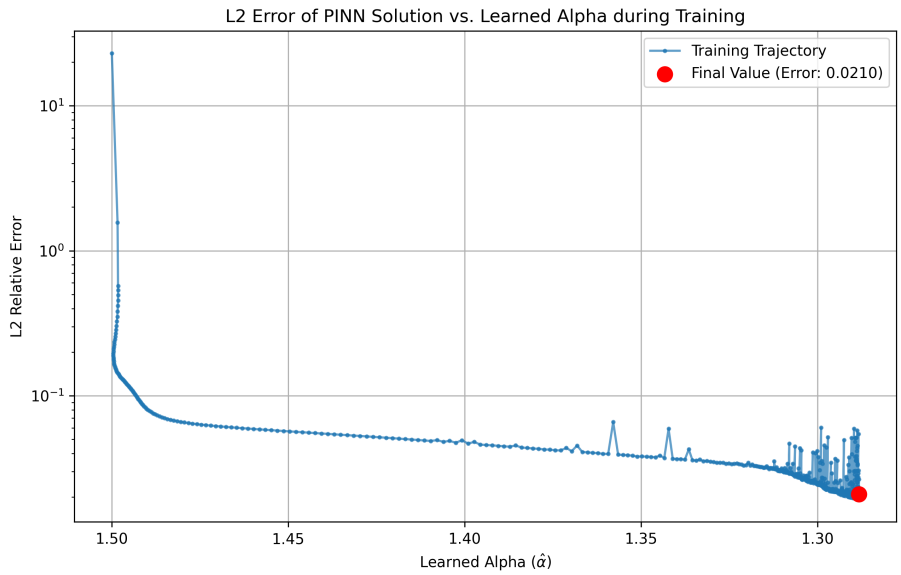
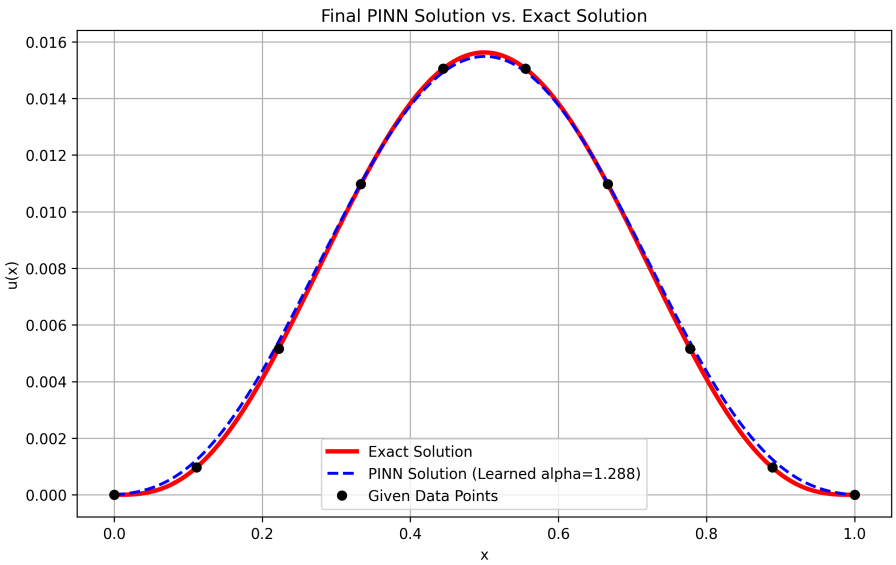


Figure 4.9

Comparison of the final cfPINN solution with the exact analytical solution for the Poisson problem.



4.3. Two-Dimensional Time-Dependent Fractional Advection-Diffusion

Building upon the one-dimensional results, we extend the framework to solve a multi-dimensional, time-dependent problem from Pang et al., 2019.

4.3.1. Problem Statement

The governing non-local equation is a fractional advection-diffusion equation on a 2D unit disk, $\Omega = \{\mathbf{x} \mid \|\mathbf{x}\|_2 \leq 1\}$:

$$\frac{\partial^\gamma}{\partial t^\gamma} u(\mathbf{x}, t) + c(-\Delta)^{\alpha/2} u(\mathbf{x}, t) = F(\mathbf{x}, t), \quad \mathbf{x} \in \Omega \tag{4.5}$$

The known analytical solution for this problem is given by (Pang et al., 2019):

$$u(\mathbf{x}, t) = (1 - \|\mathbf{x}\|_2^2)_+^{1+\alpha/2} e^{-t} \tag{4.6}$$

The corresponding forcing term $F(\mathbf{x}, t)$ is derived by applying the non-local operator to this solution with ground-truth parameters $\alpha = 1.5$ and $\gamma = 0.8$, yielding:

$$F(\mathbf{x}, t) = -t^{1-\gamma} E_{1,2-\gamma}(-t)(1 - \|\mathbf{x}\|_2^2)_+^{1+\frac{\alpha}{2}} + c \cdot 2^\alpha \Gamma(\frac{\alpha}{2} + 2)\Gamma(\frac{2+\alpha}{2}) \left[1 - (1 + \frac{\alpha}{2})\|\mathbf{x}\|_2^2\right] e^{-t} \tag{4.7}$$

where $E_{a,b}(\cdot)$ is the Mittag-Leffler function. The objective is to solve the forward problem by learning an effective spatial order $\hat{\alpha}$ for our local surrogate operator, using sparse data from the true non-local solution.

4.3.2. Methodology and Hyperparameters

The temporal derivative is handled using the 1D Caputo-type conformable operator, while the spatial fractional Laplacian is approximated using our directional conformable operator, with the integral estimated via a Monte Carlo method. Boundary conditions are enforced with the ansatz $\hat{u}(\mathbf{x}, t) = (1 - \|\mathbf{x}\|_2^2) \cdot NN(\mathbf{x}, t; \theta)$. The temporal order $\gamma = 0.8$ is held fixed. The hyperparameters are detailed in Table 4.3.

Parameter	**Value**
Neural Network Architecture	7 hidden layers, 64 neurons/layer
Optimizer	Adam
Learning Rate	1×10^{-2}
Training Epochs	1,000
Data Points (N_{data})	100
Data Loss Weight (λ)	100
Ground-Truth Orders	$\alpha = 1.5$, $\gamma = 0.8$
Learned Parameter	Effective spatial order $\hat{\alpha} \in (1, 2)$

Table 4.3
Hyperparameters for the 2D time-dependent problem with a learnable spatial order.

4.3.3. Results and Discussion

We find that the learned effective order $\hat{\alpha}$ converges to a value distinct from the true order α (Figure 4.10), and that the L2 error decreases as the learned order converges (Figure 4.11). The cfPINN solution accurately captures the spatial structure of the true solution, demonstrating the effectiveness of the local surrogate model in approximating the non-local fractional PDE.

A visual comparison of the final solution at a time slice of $t = 0.5$ is presented in Figure 4.12. The solution generated by the trained cfPINN (Figure 4.12a) can be compared to the true analytical solution (Figure 4.12b). The absolute point-wise error between the two is shown in Figure 4.12c. The final L2 relative error is on the order of 10^{-1}. For comparison, fPINN-based approaches for multi-dimensional problems have reported errors in the range of 10^{-3} to 10^{-4} (Pang et al., 2019), which again underscores the trade-off of the surrogate modeling error in our approach.

Figure 4.10

Convergence of the learned fractional order $\hat{\alpha}$ for the 2D problem.

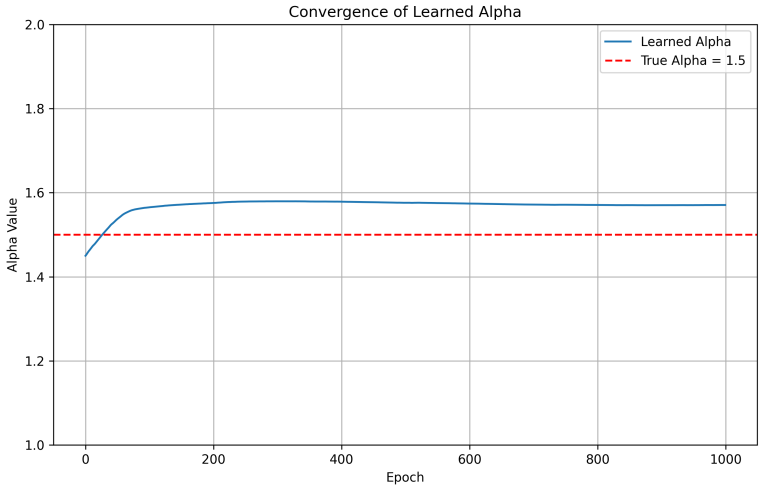


Figure 4.11

The L2 relative error of the PINN solution plotted against the learned spatial order $\hat{\alpha}$.

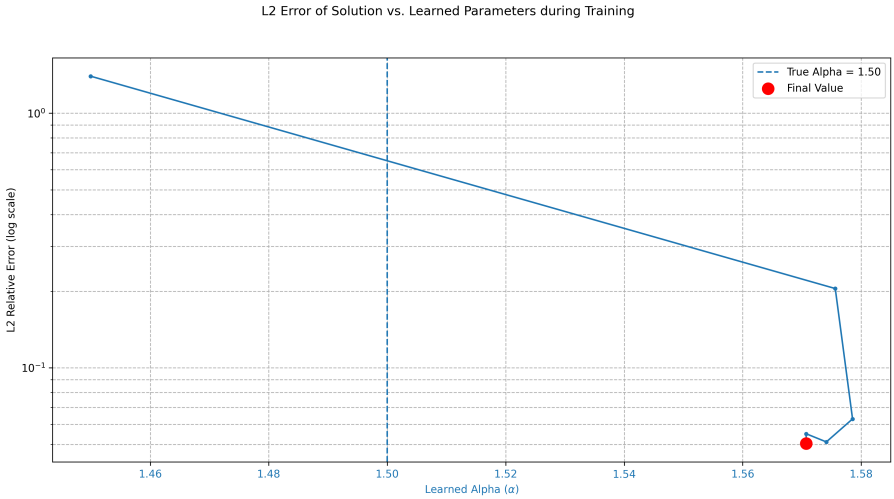
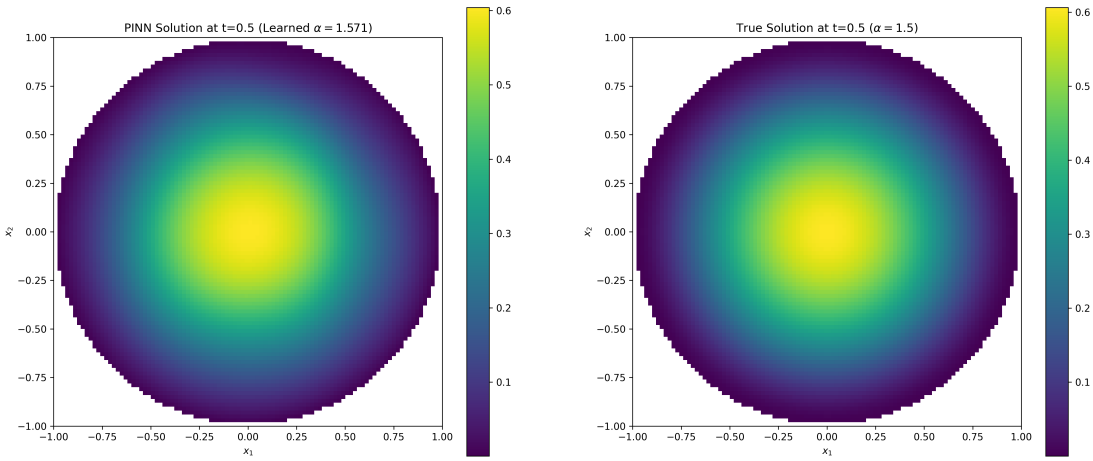
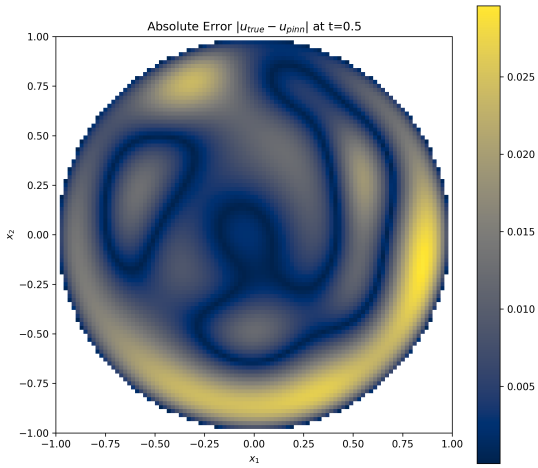


Figure 4.12

Visual comparison of the final PINN solution against the analytical solution for the 2D problem at a time slice of $t = 0.5$.



(a) *cfPINN Solution ($\hat{\alpha}$ learned)* **(b)** *True Analytical Solution ($\alpha = 1.5$)*



(c) *Absolute Point-wise Error*

5. SUMMARY AND CONCLUSIONS

Our approach to solving FDEs removes the discretization error present in fPINNs/ FDM methods, however it introduces a surrogate modeling error. This trade-off is justified by the significant computational efficiency gains, particularly in high-dimensional problems. The surrogate modeling error can be mitigated by learning an effective fractional order, as demonstrated in our numerical simulations.

The central challenge in solving FDEs within a PINN framework lies in repeatedly evaluating the differential operator to compute the loss residual. Our methodology overcomes this barrier through its use of Automatic Differentiation (AD), the fractional operator is implemented using differentiable primitives, which allows it to leverage the speed of AD. The efficiency gains of AD are well-documented: the computational cost of FDM methods is proportional to $n \cdot ops(f)$, where n is the number of input variables and $ops(f)$ is the cost of evaluating f (Margossian, 2019). Meanwhile, the cost of AD is proportional to $ops(f)$. Thus, fPINNs become computationally prohibitive for high dimensional FDEs. The choice between our method (cfPINN) and existing fractional PINN (fPINN) approaches involves a set of trade-offs, summarized in Table 5.1.

Feature	**cfPINN (Our Method)**	**fPINN**
Scalability	**+** Scalable to high dimensions	**-** Suffers from curse of dimensionality
Inverse Problems	**-** Not effective in inverse problems	**+** Can be applied to inverse problems
Function Smoothness	**-** Requires $f \in C^1$	**+** Can handle non-smooth functions
Error Source 1	**+** No discretization error	**-** Introduces discretization error
Error Source 2	**-** Inherent surrogate modeling error	**+** Directly models non-local operator

Table 5.1
A comparison of advantages (+) and disadvantages (-) of the proposed cfPINN method against the fPINN approach.

The surrogate modeling error in our method, while a limitation, can be remediated by learning an effective fractional order $\hat{\alpha}$, as explored in our work. Furthermore, while our current AD-based implementation requires the function f to be at least once-differentiable, we note that a true Conformable Fractional AD framework could, given the locality and validity of the chain rule of conformable derivatives, extend this approach to non-smooth functions.

BIBLIOGRAPHY

Abdeljawad, T. (2015). On conformable fractional calculus. *Journal of Computational and Applied Mathematics*, *279*, 57–66.

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. https://arxiv.org/abs/1502.05767

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/jax-ml/jax

Cuomo, S., di Cola, V. S., Giampaolo, F., Rozza, G., Raissi, M., & Piccialli, F. (2022). Scientific machine learning through physics-informed neural networks: Where we are and what's next. https://arxiv.org/abs/2201.05624

Diethelm, K. (2010). *The analysis of fractional differential equations: An application-oriented exposition using differential operators of caputo type* (Vol. 2004). Springer Science & Business Media.

Gao, F., & Chi, C. (2020). Improvement on conformable fractional derivative and its applications in fractional differential equations. *Journal of Function Spaces*, *2020*.

Griewank, A., & Walther, A. (2008). *Evaluating derivatives* (Second). Society for Industrial; Applied Mathematics. https://doi.org/10.1137/1.9780898717761

Guo, L., Wu, H., Yu, X., & Zhou, T. (2022). Monte carlo fpinns: Deep learning method for forward and inverse problems involving high dimensional fractional partial differential equations. *Computer Methods in Applied Mechanics and Engineering*, *400*, 115523.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural networks*, *2*(5), 359–366.

Katugampola, U. N. (2014). A new fractional derivative with classical properties. *arXiv preprint arXiv:1410.6535*.

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Lischke, A., Pang, G., Gulian, M., Song, F., Glusa, C., Zheng, X., Mao, Z., Cai, W., Meerschaert, M. M., Ainsworth, M., et al. (2019). What is the fractional laplacian? a comparative review with new results.

Magin, R. L. (2010). Fractional calculus in bioengineering, part 1-3. *Critical Reviews in Biomedical Engineering*, *32*(1-2).

Margossian, C. C. (2019). A review of automatic differentiation and its efficient implementation. *WIREs Data Mining and Knowledge Discovery*, *9*(4). https://doi.org/10.1002/widm.1305

Martínez, F., Martínez, I., Kaabar, M. K. A., & Paredes, S. (2021). On conformable laplace's equation. *Mathematical Problems in Engineering*, *2021*. https://doi.org/10.1155/2021/5514535

Metzler, R., & Klafter, J. (2000). The random walk's guide to anomalous diffusion: A fractional dynamics approach. *Physics reports*, *339*(1), 1–77.

Nocedal, J. (1980). Updating quasi-newton matrices with limited storage. *Mathematics of computation*, *35*(151), 773–782.

Pang, G., Lu, L., & Karniadakis, G. E. (2019). Fpinns: Fractional physics-informed neural networks. *SIAM Journal on Scientific Computing*, *41*(4), A2603–A2626.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., . . . Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. https://arxiv.org/abs/1912.01703

Podlubny, I. (1998). *Fractional differential equations: An introduction to fractional derivatives, fractional differential equations, to methods of their solution and some of their applications* (Vol. 198). Elsevier.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536.

Scalas, E. (2006). The application of continuous-time random walks in finance and economics. *Physica A: Statistical Mechanics and its Applications*, *362*(2), 225–239.

Tarasov, V. E. (2016). On chain rule for fractional derivatives. *Communications in Nonlinear Science and Numerical Simulation*, *30*(1-3), 1–4.

Verma, A. (2000). An introduction to automatic differentiation. *Current Science*, *78*(7), 804–807. Retrieved August 12, 2025, from http://www.jstor.org/stable/24103956